

Програмування мовою C++. Символи і рядки символів. Стандартні функції для обробки рядків символьних даних. Рядки string.

1. Типи рядків у C++

1.1. C-style рядки (масиви символів)

Це **масив char**, який закінчується спеціальним символом `'\0'`.

Використовуються у мові C та в C++ через `<cstring>`.

```
char text[] = "Hello"; // довжина масиву 6 (5 символів + '\0')
```

При копіюванні та операціях відповідальність за коректність розміру — на програмісті.

Немає контролю виходу за межі.

Працюють як звичайні масиви.

1.2. Рядки типу `std::string`

Клас зі стандартної бібліотеки C++.

Безпечні, автоматично керують пам'яттю.

Мають велику кількість зручних методів.

```
#include <string>
string s = "Hello";
```

2. Оголошення та ініціалізація символьних рядків

Приклади C-style оголошень:

```
char s1[] = "abcdef"; // автоматичне додавання '\0'
char s2[20] = "abc"; // решта елементів залишаються порожніми/сміттям
char s3[20] = {'a','b','c','\0'};
```

△ Без `'\0'` — це *не* рядок, а просто масив символів.

3. Звертання до символів

3.1. C-style

Індексний доступ:

```
char s[] = "Hello";  
cout << s[1];    // 'e'
```

Доступ через вказівники:

```
char *p = s;  
while (*p != '\0') cout << *p++;
```

3.2. std::string

```
string s = "Hello";  
char c = s[2];  
char d = s.at(2); // з перевіркою
```

4. Ввід/вивід символьних рядків

4.1. C-style

Ввід одним словом:

```
char name[255];  
cin >> name; // зупиняється на пробілі
```

Ввід рядка з пробілами:

```
cin.getline(name, 255);
```

4.2. std::string

Ввід рядка з пробілами:

```
string text;  
getline(cin, text);
```

5. Бібліотека <string.h> — функції для C-style рядків

Найважливіші функції

Функція	Призначення
strcpy(dest, src)	копіювання рядка
strncpy(dest, src, n)	копіювання n символів
strcat(dest, src)	додавання рядка
strncat(dest, src, n)	додавання n символів
strcmp(s1, s2)	порівняння рядків
strncmp(s1, s2, n)	порівняння n символів
strlen(s)	довжина рядка
strchr(s, c)	перше входження символу
strrchr(s, c)	останнє входження
strstr(s1, s2)	пошук підрядка
strtok(s, limit)	розбиття на лексеми
strdup(s)	копія рядка в динамічній пам'яті

6. Приклади операцій з C-style рядками

6.1. Видалення символу з рядка

Через індекси:

```
while (s[k] != '\0')  
    if (s[k] != '*') s[n++] = s[k++];  
s[n] = '\0';
```

Через вказівники:

```
for (pk = pn = s; *pk != '\0'; pk++)  
    if (*pk != '*') *pn++ = *pk;  
*pn = '\0';
```

6.2. Видалення підрядка між двома '#'

```
p1 = strchr(s, '#');  
p2 = strchr(s, '#');  
strcpy(p1, p2 + 1);
```

6.3. Виділення слів (лексем)

Небезпечний варіант:

```
token = strtok(str, " ,.;" );
```

Безпечний варіант (strtok_s):

```
char* context;  
token = strtok_s(str, " ", &context);
```

7. Бібліотека <ctype.h> — робота з окремими символами

Основні функції (табл. 1):

Функція	Перевіряє/перетворює
isalpha(c)	літера
isdigit(c)	цифра
isalnum(c)	буква або цифра
isspace(c)	пробільний символ
islower(c)	мала літера
isupper(c)	велика літера
tolower(c)	робить символ маленьким
toupper(c)	робить символ великим

Приклад:

```
if (isdigit(c)) cout << "Digit";
```

8. Клас `std::string` — сучасний спосіб роботи з рядками

8.1. Основні операції

Ініціалізація:

```
string s1 = "Hello";  
string s2("World");  
string s3(5, 'a'); // "aaaaa"
```

Конкатенація:

```
s = s1 + s2;
```

Додавання:

```
s.push_back('!');  
s.append(" C++");
```

Розмір:

```
s.length(); s.size();
```

Пошук і підрядки:

```
s.find("World");  
s.substr(0, 5);
```

Вставка, видалення, заміна:

```
s.insert(5, ",");  
s.erase(5, 1);  
s.replace(6, 5, "C++");
```

Порівняння:

```
s1 == s2;  
s1.compare(s2);
```

Перетворення:

stoi("123"); // → 123

to_string(45); // "45"

9. Порівняння C-style vs std::string

Можливість	C-style	std::string
автоматична пам'ять	✗	✓
захист від виходу за межі	✗	✓
легко додавати/видаляти	✗	✓
прості методи роботи	✗	✓
швидкодія	✓	✓ (практично однакова)

Висновок: сучасні програми використовують std::string, C-style потрібен для низькорівневих операцій або сумісності з C.